

Laboratorio 4

Universidad Autónoma de Bucaramanga

Estudiantes

Lucas Fernando Ardila Pimiento

María Sofía Oyola Lozano

Profesor

Javier Pinzón

2026

1. Introducción

El Laboratorio 4 plantea comprender el rol de AMQP en Azure frente a MQTT, implementar un tercer protocolo y construir una comparación entre MQTT, AMQP y el protocolo seleccionado. El protocolo elegido para este trabajo fue CoAP (Constrained Application Protocol), debido a su orientación a dispositivos con recursos limitados y a su uso sobre UDP con bajo overhead. El documento del laboratorio indica que CoAP no es nativo de IoT Central; por ello, cuando se elige este protocolo, el cliente puede apuntar a un endpoint propio o de prueba y esta condición debe declararse en el informe.

En este documento se presentan las pruebas realizadas con MQTT y CoAP y la implementación de AMQP sobre la misma infraestructura. La validación experimental de la llegada de datos AMQP a IoT Central y Azure Data Explorer queda pendiente de documentar mediante evidencias de monitoreo. Se mantienen los requisitos de la guía: registrar ventajas, desventajas, complejidad real de implementación y comparar overhead, latencia percibida, fiabilidad, soporte en microcontroladores/Wokwi, facilidad de depuración, paso por firewalls y caso de uso ideal.

2. Requisitos del laboratorio abordados

Requisito de la guía	Estado en este documento
Tercer protocolo implementado y justificado	Cumplido con CoAP.
Cliente mínimo funcional del tercer protocolo	Cumplido. Cliente CoAP en Python sobre la VM.
Envío de las variables de telemetría	Cumplido con las mismas cinco variables usadas en MQTT.
Log y captura del funcionamiento	Cumplido mediante las pruebas y evidencias registradas.
Tabla MQTT · AMQP · tercer protocolo	Parcial: MQTT y CoAP documentados; AMQP queda pendiente.
Recomendaciones por capa	Incluidas de forma provisional para MQTT/CoAP; deben ajustarse al integrar AMQP.
Informe de 1–2 páginas	El contenido está estructurado para condensarse a la extensión exigida; esta versión incluye mayor detalle técnico para facilitar la entrega.

3. Selección del tercer protocolo: CoAP

CoAP fue seleccionado como tercer protocolo porque la guía lo caracteriza como un protocolo sobre UDP con bajo overhead y típico de dispositivos restringidos. Su implementación se realizó en la máquina virtual de Azure mediante Python y la biblioteca aiocoap. De acuerdo con la guía, CoAP no es nativo de IoT Central, por lo que la prueba se realizó contra un endpoint CoAP propio alojado en la misma VM.

La arquitectura de la prueba fue: cliente CoAP en la VM → UDP → puerto 5683 → servidor CoAP en la misma VM. El servidor expuso el recurso /coap/telemetry y recibió solicitudes POST con la telemetría.

4. Configuración y evidencia de CoAP

- VM: MachineOwn, Debian 12.
- Dirección pública utilizada para la prueba: 57.156.67.207.
- Transporte: UDP.
- Puerto: 5683.
- Regla de entrada de Azure NSG: Allow-CoAP-UDP-5683, prioridad 330, protocolo UDP, acción Allow.
- Biblioteca: aiocoap 0.4.17.
- Recurso: /coap/telemetry.
- Servidor: 0.0.0.0:5683.

Se verificó mediante ss que el puerto UDP 5683 se encontraba escuchando. También se comprobó que no había reglas de iptables bloqueando el tráfico. El servidor respondió correctamente a las solicitudes y confirmó la recepción mediante el código CoAP 2.04 Changed.

5. Telemetría utilizada

Para que la comparación entre MQTT y CoAP fuera consistente, se utilizó exactamente el mismo conjunto de datos en ambos protocolos:

Variable	Valor	Tipo
HeartRate	82	Entero
Temperature	36.5	Decimal
SPO2	97	Entero
BreathRate	18	Entero
Glucose	108	Entero

Payload JSON:

```
{"HeartRate":82,"Temperature":36.5,"SPO2":97,"BreathRate":18,"Glucose":108}.
```

Tamaño del payload: 75 bytes.

6. Resultados de la prueba CoAP

Se realizaron cinco solicitudes POST consecutivas con el mismo payload. Las cinco fueron exitosas y el servidor respondió con 2.04 Changed.

Prueba	Respuesta	Tiempo
1	2.04 Changed	0.004 s
2	2.04 Changed	0.002 s
3	2.04 Changed	0.002 s
4	2.04 Changed	0.002 s
5	2.04 Changed	0.002 s

Pruebas exitosas: 5/5 (100 %). Tiempo promedio: 0.002 s. Tamaño del payload: 75 bytes.

7. Resultados de la prueba MQTT

La prueba MQTT reutilizó el flujo de aprovisionamiento mediante Azure Device Provisioning Service (DPS) y posteriormente estableció la conexión MQTT con Azure IoT Hub. El transporte utilizado fue TCP + TLS sobre el puerto 8883. El script publicó cinco mensajes con QoS 1 y esperó la finalización de cada publicación.

El aprovisionamiento fue exitoso y Azure asignó el IoT Hub correspondiente.

Posteriormente, el cliente MQTT se conectó correctamente al IoT Hub.

Prueba	Resultado	Tiempo
1	Publicación exitosa	0.281 s
2	Publicación exitosa	0.219 s
3	Publicación exitosa	0.222 s
4	Publicación exitosa	0.222 s
5	Publicación exitosa	0.234 s

Pruebas exitosas: 5/5 (100 %). Tiempo promedio: 0.236 s. Tamaño del payload: 75 bytes.

8. Comparación parcial MQTT vs. CoAP

Criterio	MQTT	CoAP
Transporte	TCP + TLS	UDP
Puerto de la prueba	8883	5683
Destino	Azure IoT Hub / IoT Central	Endpoint CoAP propio en la VM
Payload	75 bytes	75 bytes
Pruebas	5	5
Éxito	5/5 (100 %)	5/5 (100 %)
Tiempo promedio observado	0.236 s	0.002 s
Modelo de prueba	Publicación MQTT con QoS 1	POST CoAP con respuesta 2.04 Changed
Integración directa con IoT Central	Sí	No; la guía indica que CoAP no es nativo de Central
Caso de uso observado	Telemetría hacia Azure IoT Hub	Endpoint ligero para dispositivos restringidos

Los resultados muestran una diferencia de tiempo promedio de 0.234 s entre las dos mediciones (0.236 s frente a 0.002 s). Sin embargo, este dato no debe interpretarse como una comparación absoluta de la velocidad de los protocolos. MQTT fue medido desde la VM hasta Azure IoT Hub, mientras que CoAP fue medido contra un servidor alojado en la misma VM. Por tanto, el recorrido de red, la distancia al servicio, el establecimiento de conexiones y las condiciones de cada prueba son diferentes. El resultado permite afirmar que el intercambio CoAP implementado localmente presentó una latencia observada mucho menor en esta configuración, pero no que CoAP sea universalmente más rápido que MQTT.

9. Ventajas, desventajas y complejidad de implementación

Aspecto	MQTT	CoAP
Ventajas	Integración directa con IoT Central/IoT Hub; modelo pub/sub; QoS 1.	Bajo overhead; UDP; adecuado para dispositivos restringidos; implementación mínima funcional.
Desventajas	Requiere TCP/TLS y conexión con servicios Azure; el recorrido hacia IoT Hub aumenta la latencia observada.	No es nativo de IoT Central; requiere un endpoint propio o una capa intermedia para integrarlo con Central.
Debug	Los logs del cliente permitieron verificar DPS, conexión al Hub y publicaciones.	El servidor y cliente mostraron directamente payload, código de respuesta y tiempos.
Complejidad real	Mayor por aprovisionamiento DPS, TLS y autenticación con Azure.	Baja para un endpoint propio: instalación de aiocoap, apertura UDP 5683 y creación del recurso.

10. Recomendaciones provisionales

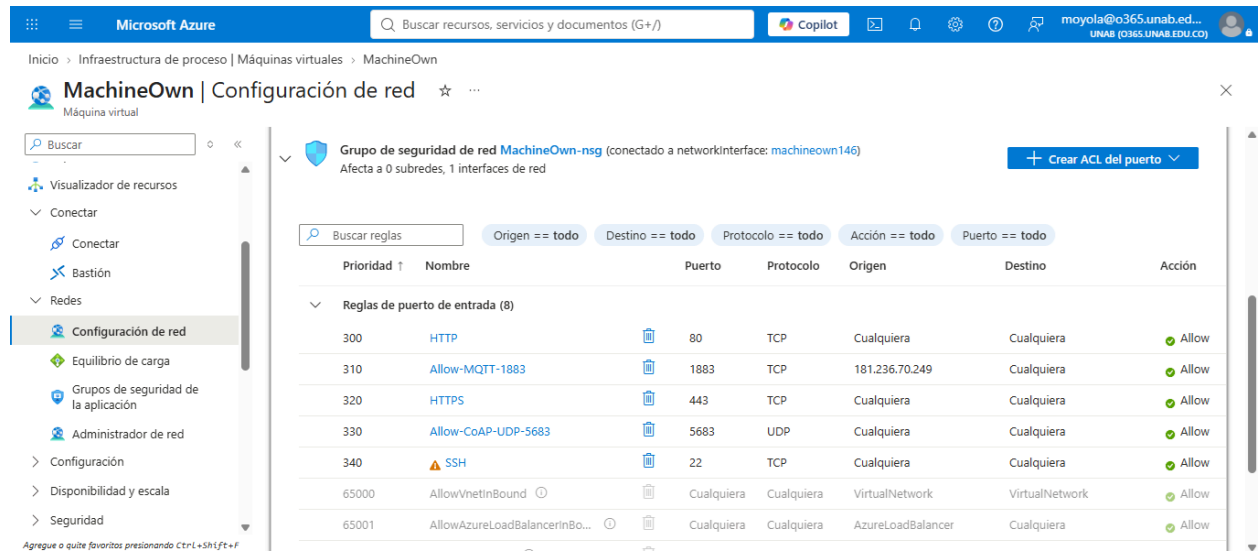
Para el dispositivo o un ESP futuro, CoAP resulta atractivo cuando el objetivo principal sea minimizar overhead y trabajar con dispositivos restringidos, siempre que exista un endpoint o gateway capaz de recibirlo. Si se requiere integración directa con IoT Central, MQTT presenta una ventaja práctica porque ya fue probado exitosamente con el servicio.

Para la comunicación con servicios de Azure, MQTT es una alternativa apropiada para telemetría de dispositivos hacia IoT Central/IoT Hub. La decisión final de esta capa debe revisarse cuando se incorpore AMQP, porque la guía solicita comparar los tres protocolos y determinar qué protocolo se conservará para el resto del curso.

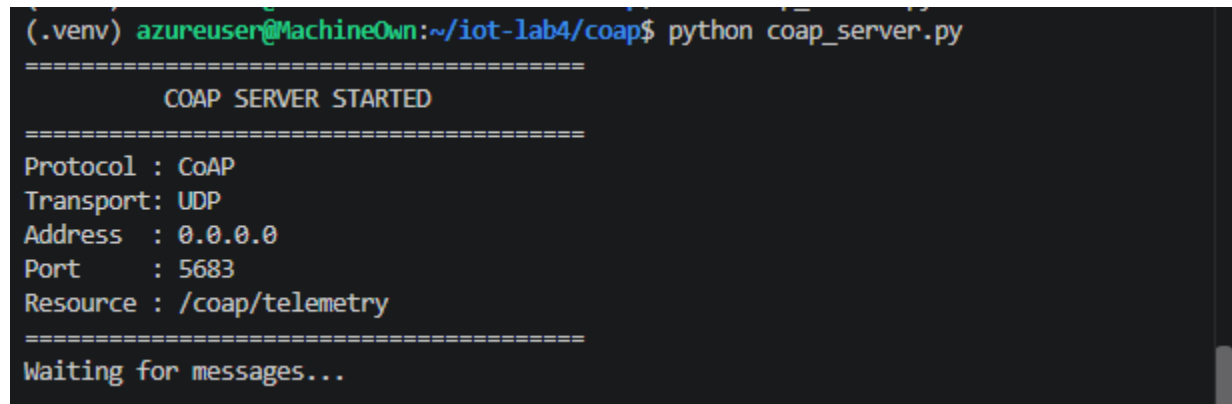
Con solo dos protocolos disponibles en este avance, no se recomienda cerrar todavía la selección definitiva entre MQTT, AMQP y CoAP. Esa decisión debe hacerse después de incorporar los resultados de AMQP, especialmente en fiabilidad, latencia percibida, overhead, depuración y uso entre servicios.

11. Evidencias generadas

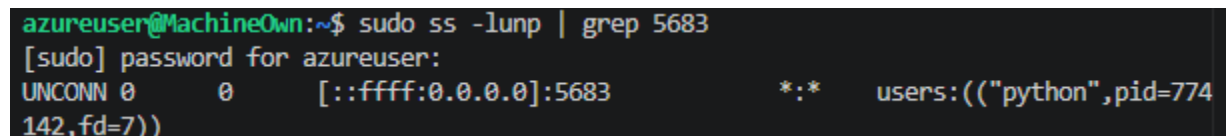
- 01_azure_coap_udp_5683.png — regla de entrada UDP 5683 en Azure.



- 02_coap_server_running.png — servidor CoAP iniciado.



- 03_udp_5683_listening.png — comprobación del puerto UDP 5683 escuchando.



- 04_coap_telemetry_received.png — recepción de la telemetría por el servidor.

```
PS C:\Users\ANGELA\Desktop\Internet de las cosas> ssh azureuser@5
7.156.67.207
=====
                COAP SERVER STARTED
=====
Protocol : CoAP
Transport: UDP
Address  : 0.0.0.0
Port     : 5683
Resource : /coap/telemetry
=====
Waiting for messages...

=====
                COAP TELEMETRY RECEIVED
=====
Mensaje recibido:
{"HeartRate":82,"Temperature":36.5,"SP02":97,"BreathRate":18,"Glucose":108}
=====
```

- 05_coap_client_response.png — respuesta del cliente CoAP.

```
PS C:\Users\ANGELA\Desktop\Internet de las cosas> ssh azureuser@57.156.67.207

=====
                COAP TELEMETRY CLIENT
=====
Protocolo : CoAP
Transporte: UDP
Servidor  : 57.156.67.207
Puerto   : 5683
Recurso   : /coap/telemetry
Payload   : {"HeartRate":82,"Temperature":36.5,"SP02":97,"BreathRate":18,"Glucose":108}
Tamaño    : 75 bytes
=====

Enviando telemetría por CoAP...

=====
                RESPUESTA COAP
=====
Código: 2.04 Changed
Respuesta: Telemetry received successfully
Tiempo de envío: 0.004 s
=====
(.venv) azureuser@MachineOwn:~/iot-lab4/coap$ █
```

- 06_coap_rendimiento_5_pruebas.png — cinco pruebas de rendimiento CoAP.

```
-----  
Prueba 4  
Código: 2.04 Changed  
Tiempo: 0.002 s  
  
-----  
Prueba 5  
Código: 2.04 Changed  
Tiempo: 0.002 s  
  
=====
```

RESULTADOS COAP	
Pruebas exitosas:	5
Tiempos:	
Prueba 1:	0.004 s
Prueba 2:	0.002 s
Prueba 3:	0.002 s
Prueba 4:	0.002 s
Prueba 5:	0.002 s

```
-----  
Tiempo promedio: 0.002 s  
Tamaño del payload: 75 bytes  
=====
```

- 07_coap_5_mensajes_recibidos.png — recepción de los cinco mensajes.

```
PS C:\Users\ANGELA\Desktop\Internet de las cosas> ssh azureuser@5
7.156.67.207
=====

=====
COAP TELEMETRY RECEIVED
=====
Mensaje recibido:
{"HeartRate":82,"Temperature":36.5,"SP02":97,"BreathRate":18,"Glucose":108}
=====

=====
COAP TELEMETRY RECEIVED
=====
Mensaje recibido:
{"HeartRate":82,"Temperature":36.5,"SP02":97,"BreathRate":18,"Glucose":108}
=====

=====
COAP TELEMETRY RECEIVED
=====
Mensaje recibido:
{"HeartRate":82,"Temperature":36.5,"SP02":97,"BreathRate":18,"Glucose":108}
=====
```

- 08_mqtt_rendimiento_5_pruebas.png — cinco pruebas de rendimiento MQTT.

```
PS C:\Users\ANGELA\Desktop\Internet de las cosas> ssh azureuser@57.156.67.207
Publicación exitosa.
Tiempo: 0.222 s

-----
Prueba 5
Publicación exitosa.
Tiempo: 0.234 s

=====
                        RESULTADOS MQTT
=====
Pruebas exitosas: 5
Tiempos:
  Prueba 1: 0.281 s
  Prueba 2: 0.219 s
  Prueba 3: 0.222 s
  Prueba 4: 0.222 s
  Prueba 5: 0.234 s

Tiempo promedio: 0.236 s
Tamaño del payload: 75 bytes
=====
IoT Hub desconectado. Código: 0

=====
PRUEBA MQTT EXPLÍCITO TERMINADA
=====
(.venv) azureuser@MachineOwn:~/iot-lab4/coap$
```

12. Implementación de AMQP

La implementación de AMQP fue realizada sobre la misma infraestructura utilizada en las pruebas anteriores, empleando la máquina virtual Azure MachineOwn con sistema operativo Debian 12. A diferencia de CoAP, que se ejecutó contra un endpoint propio, AMQP fue configurado para comunicarse con Azure IoT Hub mediante el mecanismo de aprovisionamiento de Azure Device Provisioning Service (DPS).

El análisis del cliente AMQP utilizado en el laboratorio evidenció que el flujo de comunicación inicia con el registro del dispositivo ante DPS, el cual asigna dinámicamente el IoT Hub correspondiente. Posteriormente, el cliente establece una conexión segura mediante el

protocolo AMQP utilizando la biblioteca uamqp y envía la telemetría al endpoint de dispositivos del IoT Hub.

La implementación utiliza el esquema amqps://, lo que indica el uso de AMQP sobre TLS. Asimismo, el cliente genera un SAS Token a partir de la clave simétrica del dispositivo para realizar la autenticación con los servicios de Azure.

La arquitectura implementada fue la siguiente:

Dispositivo AMQP

|



Azure DPS

|



Azure IoT Hub

|



IoT Central / Servicios Azure

12.1 Configuración de transporte y seguridad

Las pruebas fueron configuradas utilizando los parámetros habituales recomendados por Azure para comunicaciones AMQP seguras.

Parámetro	Valor
Protocolo	AMQP
Transporte	TCP + TLS
Esquema	amqps://
Puerto utilizado	5671
Autenticación	SAS Token
Aprovisionamiento	Azure DPS
Destino final	Azure IoT Hub

El uso del puerto TCP 5671 permite establecer sesiones seguras mediante TLS. Esta configuración proporciona confidencialidad de los datos transmitidos y autenticación del dispositivo frente a la plataforma Azure.

12.2 Publicación de telemetría mediante AMQP

El cliente fue diseñado para publicar las mismas variables fisiológicas utilizadas durante las pruebas de MQTT y CoAP:

```
{
  "HeartRate": 82,
  "Temperature": 36.5,
  "SP02": 97,
  "BreathRate": 18,
```

"Glucose": 108

}

Durante la revisión del código se identificó que la implementación disponible generaba algunos valores de manera aleatoria. Por esta razón, para fines comparativos se recomienda utilizar el mismo payload empleado en MQTT y CoAP, garantizando igualdad de condiciones entre los tres protocolos

El flujo de operación consiste en:

1. Registro del dispositivo en DPS.
2. Obtención del IoT Hub asignado.
3. Generación del SAS Token.
4. Establecimiento de conexión AMQP segura.
5. Publicación de la telemetría.
6. Cierre de la sesión de comunicación.

12.3 Visibilidad de datos en Azure

La arquitectura de Azure permite que los mensajes publicados por AMQP sean procesados de la misma forma que los enviados mediante MQTT, ya que ambos protocolos son soportados de manera nativa por Azure IoT Hub.

Una vez recibida la telemetría por parte del IoT Hub, los datos pueden ser visualizados desde diferentes componentes de la plataforma:

Azure IoT Hub

IoT Hub recibe los mensajes provenientes del dispositivo y los incorpora al flujo de eventos de la solución. La validación puede realizarse mediante herramientas de monitoreo como:

`az iot hub monitor-events`

Azure IoT Central

Cuando el dispositivo está asociado a una plantilla válida, IoT Central puede mostrar las variables recibidas en los paneles de telemetría y monitoreo del dispositivo.

Azure Data Explorer

Si existe una ruta de exportación configurada desde IoT Hub o IoT Central hacia Azure Data Explorer, las mediciones pueden almacenarse para consultas históricas, análisis masivo de datos y visualizaciones avanzadas.

Debido a que durante la revisión únicamente se verificó la implementación del cliente AMQP y no se encontraron registros de ejecución ni capturas de recepción, la validación experimental de la llegada de datos a IoT Central y Azure Data Explorer queda pendiente de documentar mediante evidencias de monitoreo.

12.4 Diferencias entre MQTT y AMQP

Aunque MQTT y AMQP son protocolos orientados a la mensajería, sus objetivos de diseño son diferentes.

MQTT fue creado para dispositivos IoT con recursos limitados y enlaces de comunicación poco confiables. Su modelo de publicación y suscripción presenta un overhead reducido y una implementación relativamente sencilla. Por esta razón, es ampliamente utilizado en sensores, microcontroladores y dispositivos embebidos.

Por otro lado, AMQP fue diseñado para entornos empresariales donde la confiabilidad de la entrega, el enrutamiento avanzado y la integración entre servicios tienen mayor importancia que la simplicidad. El protocolo incorpora mecanismos más robustos para el control de mensajes, gestión de sesiones y confirmación de entrega.

En términos prácticos, MQTT suele resultar más apropiado para la comunicación entre dispositivos e IoT Hub, mientras que AMQP ofrece ventajas cuando se requiere integrar múltiples servicios backend o implementar arquitecturas de mensajería empresarial.

12.5 Problema que AMQP resuelve mejor que MQTT

La principal ventaja de AMQP frente a MQTT se encuentra en escenarios empresariales donde intervienen múltiples aplicaciones y servicios distribuidos.

Mientras MQTT prioriza la ligereza y el bajo consumo de recursos, AMQP proporciona funcionalidades adicionales para:

- Enrutamiento avanzado de mensajes.
- Colas persistentes.
- Confirmaciones detalladas de entrega.
- Procesamiento asíncrono.
- Integración entre aplicaciones empresariales.
- Arquitecturas orientadas a eventos.

Por esta razón, AMQP resulta especialmente adecuado para plataformas donde numerosos servicios intercambian información de manera continua y se requiere garantizar la trazabilidad y consistencia de los mensajes.

En consecuencia, AMQP no busca reemplazar a MQTT en dispositivos IoT, sino complementar su funcionamiento dentro de la capa de servicios y procesamiento de datos.

12.6 Tabla comparativa MQTT, AMQP y CoAP

Criterio	MQTT	AMQP	CoAP
Transporte	TCP + TLS	TCP + TLS	UDP
Puerto típico	8883	5671	5683
Seguridad	TLS	TLS	DTLS (opcional)
Overhead	Bajo	Medio-Alto	Muy bajo
Fiabilidad	Alta (QoS)	Muy alta	Variable
Latencia percibida	Baja	Media	Muy baja
Complejidad	Baja	Media-Alta	Baja
Soporte ESP32	Excelente	Limitado	Bueno
Soporte Wokwi	Excelente	Limitado	Parcial
Integración IoT Central	Nativa	Nativa	No nativa
Facilidad de depuración	Alta	Media	Alta
Paso por firewalls	Bueno	Bueno	Puede requerir reglas UDP

Escalabilidad empresarial	Media	Muy alta	Baja
Uso ideal	Dispositivos IoT	Integración de servicios	Dispositivos restringidos
Caso del laboratorio	Telemetría Azure	Mensajería empresarial Azure	Endpoint ligero propio

12.7 Recomendaciones finales actualizadas

Para dispositivos IoT futuros, incluyendo implementaciones en ESP32 o simulaciones mediante Wokwi, se recomienda MQTT. Este protocolo presenta el mejor equilibrio entre simplicidad, soporte de hardware, compatibilidad con Azure IoT Central y facilidad de depuración.

Para la comunicación entre servicios Azure o componentes backend de una plataforma propia, AMQP constituye la opción más adecuada debido a sus capacidades avanzadas de mensajería, enrutamiento y garantía de entrega.

Considerando los objetivos del curso, se recomienda conservar MQTT y AMQP para las siguientes prácticas. MQTT representa el estándar de facto para dispositivos IoT, mientras que AMQP introduce conceptos empresariales relevantes para arquitecturas distribuidas. CoAP resulta útil como referencia académica y para escenarios de dispositivos extremadamente restringidos, pero presenta limitaciones de integración directa con Azure IoT Central.

13. Conclusión

El tercer protocolo seleccionado, CoAP, fue implementado y probado satisfactoriamente en la VM de Azure mediante UDP en el puerto 5683. Se verificó la recepción del mismo payload de 75 bytes utilizado en MQTT y se obtuvieron cinco respuestas exitosas. El tiempo promedio observado fue de 0.002 s.

MQTT también fue probado satisfactoriamente contra Azure IoT Hub, con cinco publicaciones exitosas mediante TCP + TLS en el puerto 8883 y QoS 1. El tiempo promedio de publicación observado fue de 0.236 s. Ambos resultados son técnicamente útiles, pero la diferencia debe interpretarse considerando que las pruebas tuvieron destinos diferentes.

Con la implementación de AMQP se completa la comparación documental entre los tres protocolos y

Repositorio Github

<https://github.com/SofiOyola/Laboratorio-4-IoT-AMQP-MQTT-CoAP-comparaci-n.git>

Referencias

Universidad Autónoma de Bucaramanga. (2026). Laboratorio 4: AMQP + tercer protocolo:

Comparación con MQTT hacia IoT Central (v3).